

Consignes de Remise — Projet Angular & .NET

Date limite de remise : 21 juin à 23h59

ATTENTION — La remise comporte deux obligations cumulatives. L'absence de l'un ou de l'autre entraîne automatiquement la note de 0, sans possibilité de rattrapage.

1. Technologies obligatoires

Le projet doit impérativement respecter les technologies et architectures vues au cours.

Backend

- ASP.NET Core (.NET)
- Architecture en couches suivant la **Clean Architecture vue au cours**
- Communication avec la base de données exclusivement via **Dapper**
- Respect des principes SOLID et Clean Code
- Séparation claire des responsabilités (Core, Infrastructure, API)

L'utilisation d'Entity Framework est interdite.

Frontend

- Angular
- Utilisation de la **nouvelle syntaxe Angular vue au cours** (`@if`, `@for`, `@switch`, etc.)
- Gestion de l'état exclusivement via les **Services Angular**
- Utilisation éventuelle d'une bibliothèque de composants UI (Angular Material, PrimeNG, etc.)

L'utilisation de bibliothèques de gestion d'état externes est interdite (NgRx, Redux, etc.).

2. Ce que vous devez remettre

#	Livrable	Canal
1	Fichier ZIP du projet complet	Moodle
2	Lien GitHub du dépôt	E-mail à l'enseignant

Les deux sont obligatoires.

L'absence de l'un des deux = **0 directement**.

3. Fichier ZIP — Dépôt Moodle

Le fichier ZIP doit contenir :

- Le projet Angular
- Le projet .NET
- Les scripts SQL nécessaires à la création de la base de données
- Le fichier README

À exclure avant compression :

- `node_modules`
- `bin`
- `obj`
- tout autre dossier généré automatiquement

Le dépôt doit être effectué avant la date limite.

4. Dépôt GitHub — Envoi par e-mail

Le dépôt GitHub doit :

- Être public
- Contenir l'historique complet du développement
- Montrer une progression cohérente du projet
- Être organisé et documenté

L'e-mail devra avoir comme objet :

[Angular/.NET] Remise projet – Nom Prénom

5. README obligatoire

Un fichier README doit être présent à la racine du dépôt GitHub ainsi que dans l'archive ZIP.

Le README doit expliquer de manière détaillée et étape par étape :

Prérequis

- Version de .NET utilisée
- Version de Node.js utilisée
- Version d'Angular utilisée
- Version du SGBD utilisé

Installation

- Création de la base de données

- Exécution des scripts SQL
- Configuration éventuelle des chaînes de connexion
- Configuration éventuelle des variables d'environnement

Lancement du Backend

Toutes les commandes nécessaires pour démarrer l'API.

Lancement du Frontend

Toutes les commandes nécessaires pour démarrer l'application Angular.

Comptes de test

Si l'application nécessite une authentification, les identifiants de démonstration doivent être fournis.

Le projet doit pouvoir être lancé uniquement à partir des instructions présentes dans le README.

Si le projet ne peut pas être exécuté correctement en suivant les instructions du README, la note sera de 0.

6. Le projet (le code)

Fonctionnement

L'application doit :

- Se lancer sans erreur
- Être démontrable le jour de l'oral
- Posséder une base de données fonctionnelle

Toute application qui ne démarre pas correctement pourra être considérée comme non fonctionnelle.

Qualité du code

Le code doit respecter :

- Les conventions de nommage
- Les principes du Clean Code
- Les principes SOLID
- La Clean Architecture vue au cours

Vous devez être capable d'expliquer l'intégralité du code présenté.

7. Contraintes techniques obligatoires

Backend

La structure minimale attendue est :

```
Solution
|
├── API
├── Core
└── Infrastructure
```

La communication avec la base de données doit obligatoirement passer par :

- Dapper
- Les repositories et patterns vus au cours

Aucun accès direct à la base de données depuis les contrôleurs ne sera accepté.

Frontend

Le projet doit démontrer l'utilisation :

- Des composants Angular
- Des services Angular
- Des formulaires Angular
- Du routage Angular
- De la nouvelle syntaxe Angular (`@if`, `@for`, `@switch`, ...)

La communication avec l'API doit être réalisée via :

- HttpClient
- Des services Angular dédiés

La gestion de l'état doit être réalisée uniquement via les services Angular.

8. Épreuve orale

L'oral se compose de trois parties.

1. Question théorique

Question portant sur :

- Angular
- ASP.NET Core
- Clean Architecture
- Dapper
- Architecture logicielle

Objectif :

Vérifier la maîtrise des concepts vus au cours.

2. Analyse du flux applicatif

Vous devrez expliquer le parcours complet d'une donnée.

Exemple :

```
Angular Component
→ Angular Service
→ API Controller
→ Core
→ Infrastructure
→ Dapper
→ Base de données
```

Puis le retour de cette donnée jusqu'à l'interface utilisateur.

Objectif :

Vérifier votre compréhension globale de l'architecture mise en place.

3. Modification de code en direct

Une modification sera demandée en direct.

Exemples :

- Ajouter une fonctionnalité
- Modifier une règle métier
- Ajouter un filtre
- Modifier une requête
- Ajouter un champ dans un formulaire

L'utilisation de l'IA (ChatGPT, Copilot, etc.) est strictement interdite pendant cette épreuve.

Objectif :

Vérifier que vous êtes l'auteur du projet et que vous maîtrisez votre environnement de développement.

9. Conditions éliminatoires

Situation	Conséquence
ZIP absent sur Moodle	0 directement
Dépôt GitHub absent	0 directement

Situation	Conséquence
Application non fonctionnelle	0 directement
README absent	0 directement
Projet impossible à lancer en suivant le README	0 directement
Utilisation d'Entity Framework	0 directement
Absence de Clean Architecture	0 directement
Utilisation d'une librairie de gestion d'état externe	0 directement

10. Niveau attendu

Le projet doit démontrer :

- Une architecture professionnelle
- Une séparation claire des responsabilités
- Une maîtrise d'Angular moderne
- Une maîtrise de ASP.NET Core
- Une utilisation correcte de Dapper
- Une compréhension complète de la Clean Architecture

L'objectif n'est pas seulement de produire une application fonctionnelle, mais de démontrer la maîtrise des concepts d'architecture et de développement abordés durant le cours.